

TurboSheet PRD

Next-Generation Rust-Powered Browser Automation for Node.js

Version 1.0 | Status: Draft | April 2026

1. Executive Summary

TurboSheet is a breakthrough browser automation framework written in Rust, specifically designed to eliminate the performance bottlenecks of existing tools like Playwright, Puppeteer, and Cypress. By utilizing native FFI (Foreign Function Interface) via `napi-rs`, TurboSheet bypasses the heavy JSON-RPC serialization overhead of current Node.js automation, delivering 3x faster execution and 50% lower memory footprint.

2. Problem Statement

- **The Serialization Tax:** Playwright/Puppeteer communicate with browsers via WebSockets using JSON-RPC. For high-frequency interactions or large DOM transfers, the serialization/deserialization becomes a CPU bottleneck.
- **Single-Threaded Constraints:** Managing multiple browser contexts in Node.js often blocks the event loop, causing latency in large-scale scraping or complex agentic workflows.
- **AI Overhead:** Modern AI Agents require frequent page-state snapshots. Extracting full HTML and passing it to LLMs is slow and token-expensive.

3. Product Objectives

- **Performance:** Outperform Playwright in script execution speed by at least 2x.
- **Versatility:** Provide a dual-core engine (Chromiumoxide for raw speed, Rustenium for stealth/BiDi).
- **Efficiency:** Reduce memory overhead by 40-60% compared to Puppeteer.
- **Native Integration:** Seamlessly integrate into the Node.js ecosystem with zero-config TypeScript support.

4. Key Features

4.1 Dual-Core Engine Architecture

TurboSheet allows users to swap the underlying driver based on the "intent" of the task:

Mode	Core Driver	Primary Use Case
Turbo (Speed)	Chromiumoxide	B2B SaaS automation, Data Scraping, E2E Testing.
Stealth (AI)	Rustenium	AI Agents, Bot-detection bypass, Multi-touch simulation.

4.2 Native Selector Engine

Unlike current tools that execute "wait-for-selector" logic in JavaScript, TurboSheet executes this logic directly in Rust. This reduces the "back-and-forth" chatter between Node and the Browser.

4.3 AI-Native State Extraction

A built-in `page.getAgentSnapshot()` method that returns a Rust-optimized Accessibility Tree or "Visual DOM" instead of raw HTML, significantly reducing token consumption for LLMs.

5. Technical Architecture

The Stack:

- **Core:** Rust 1.75+ (Tokio for async management).
- **Bridge:** `napi-rs` for high-performance Node.js bindings.
- **Drivers:** `chromiumoxide` (CDP) & `rustenium` (WebDriver BiDi).
- **Distribution:** Pre-compiled native binaries for Linux, macOS, and Windows.

The architecture moves the "Brain" of the automation from the Node.js Event Loop to a multi-threaded Rust Runtime. Node.js only acts as the orchestrator.

6. Target Segments & Use Cases

- **Enterprise SaaS (HRIS/Procurement):** Automating complex internal workflows that require high reliability and speed.
- **Web Scraping at Scale:** Reducing server costs for high-throughput data extraction via memory-efficient Rust threads.
- **AI Agent Developers:** Teams building autonomous agents that need real-time browser feedback with minimal latency.

7. Roadmap

Phase 1: MVP	Core NAPI-RS bridge + Chromiumoxide support + Basic CRUD actions.
Phase 2: Stealth	Integration of Rustenium core + Human-like input simulation (Bezier curves).
Phase 3: AI-Opt	Advanced state-compression and Accessibility Tree snapshots for LLMs.
Phase 4: Ecosystem	Playwright-migration CLI tool + Cloud-grid support.

TurboSheet - Built for the Speed of Rust, with the Ergonomics of TypeScript.